WEB DEVELOPMENT 1

Learning Groups: P107, P108, E101, P102

Lecturer: Mr. Maina Samuel

week 7 **learning topics**

- **7.0 Introduction to server- side scripting using PHP**
- **7.1 PHP variables, PHP If, If else, else if. PHP for, while, do while loops**
- **7.2 PHP Arrays**
- **7.3 Working with Forms using HTTP GET & POST methods**
- **7.4 How to Create a Login Form with in PHP and MySQL?**

7.0 Introduction to server- side scripting using PHP

PHP (Hypertext Pre-processor) is a web server-side programming language. It enables developers to build web applications that interact with databases.

PHP isn't the only code that involves server-side scripting, and server-side scripting isn't limited to websites. Other server-side programming languages are Python, Ruby, C#, C++, and Java. There are many instances of server-side scripting, which provides a customized experience for users.

Why PHP?

- **Easy to learn:** PHP is easy to learn and use. For beginner programmers who just started out in web development, PHP is often considered as the preferable choice of language to learn.
- **Open source:** PHP is an open-source project. It is developed and maintained by a worldwide community of developers who make its source code freely available to download and use.
- **Portability:** PHP runs on various platforms such as Microsoft Windows, Linux, Mac OS, etc. and it is compatible with almost all servers used today such Apache, IIS, etc.
- **Fast Performance:** Scripts written in PHP usually execute or runs faster than those written in other scripting languages like ASP, Ruby, Python, Java, etc.
- **Vast Community:** Since PHP is supported by the worldwide community, finding help or documentation related to PHP online is extremely easy.

Here are some of the most popular PHP frameworks:

- Yii 2 – A generic PHP framework used when developing web applications with many ties.
- Laravel – A PHP framework designed to make web apps easier and faster for developers.
- FuelPHP – A PHP framework that supports data-oriented web applications.

Install PHP

To install PHP, we will suggest you to install AMP (Apache, MySQL, PHP) software stack. It is available for all operating systems. There are many AMP options available in the market that are given below:

- **WAMP** for Windows **LAMP** for Linux, **MAMP** for Mac **SAMP** for Solaris
- **XAMPP** (Cross, Apache, MySQL, PHP, Perl) for Cross Platform: It includes some other components too such as FileZilla, OpenSSL, Webalizer, Mercury Mail, etc.

History of PHP

Rasmus Lerdorf wrote the first version of PHP in 1994. The language was built from the C language as a means of replacing the snippets of Perl that he had been using on his personal homepage. However, it wasn't until 1995 that he released the first formal and public version of the language. At this time PHP referred to Personal Homepage Tools. JavaScript would be released as a client-side language in 1996, so PHP's release as a server-side language around speaks to the robust growth in the tools of the Internet that occurred during this time.
among the **most popular uses of PHP** occurred in regards **to Mark Zuckerberg's use of this language in 2003–2004** when he was developing Facebook out of his Harvard dorm room. Since this time, **Facebook has developed its own language Hack** that is a version of PHP that scales efficiently.

Download and Install XAMPP Server

Click me to download XAMPP server



PHP Syntax Rules

Below we have a listed down the main syntax rules that you must follow while writing php code.

1. All the php code in a php script should be enclosed within <?php and ?>, else it will not be considered as php code. Adding php code inside the PHP tags is known as **Escaping to php**.

```
<?php  ...  ?>
```

Apart from the standard <?php and ?>, you can also use the **Short-open tags**:

```
<?  ...  ?>
```

Or use the HTML script tags, like we do for adding JavaScript code in HTML document:

```
<script language="PHP">  ...  </script>
```

2. Every expression in PHP ends with a **semicolon**;

3. **Commenting PHP code**: Both single line and multi-line comments are supported in PHP. For single line comment, we can either use # or // before the comment line. For example,

```php
<?php
    # This is also a single line comment
    # another line of comment

    // This is a single line comment
    echo "Example for Single line Comments";
?>
```

4. **PHP is case sensitive**, which means that a variable $tiger is not same as $Tiger. Both of these, represent two different variables here. But all the predefined keywords and functions like if, else, echo etc are case insensitive.

```php
<?php

    echo "Hello, World!";

    ECHO "Hello, World!";

?>
```

```
Hello, World!

Hello, World!
```

5. PHP uses **curly braces to define a code block**.

```php
<?php
    if($zero == 0)
    {
        echo "If condition satisfied";
        echo "This is a code block";
    }
?>
```

7.1 PHP variables, PHP If, If else, else if. PHP for, while, do while loops

Variables in PHP

The name given to the memory space is called a **Variable.**

```php
<?php
    $variableName = value;
?>
```

Rules for creating Variables in PHP

Here we have a few basic rules that you must keep in mind while creating variables in PHP. All the rules are explained with help of simple examples.

1. A variable name will always start with a **$** sign, followed by the variable name.
2. A variable name should not start with a numeric value. It can either start with an alphabet or an underscore sign _.

```php
<?php
    $var = "I am a variable";
    $_var = 5;
    // Invalid variable name
    $7var = "I am a variable too";
?>
```

PHP Variable: Declaring string, integer, and float

```php
<?php
$str="hello string";
$x=200;
$y=44.6;
echo "string is: $str <br/>";
echo "integer is: $x <br/>";
echo "float is: $y <br/>";
?>
```

Output :

```
string is: hello string
integer is: 200
float is: 44.6
```

PHP Variable: Sum of two variables

```php
<?php
$x=5;
$y=6;
$z=$x+$y;
echo $z;
?>
```

output : 11

PHP has three types of variable scopes:

- Local variable
- Global variable
- Static variable

Local variable

The variables that are declared within a function are called **local variables** for that function. These local variables have their scope only in that particular function in which they are declared. This means that these variables cannot be accessed outside the function, as they have local scope.

Local variables are needed for encapsulation, which is a good practice in programming.

```php
<?php
   function local_var()
   {
      $num = 45;  //local variable
      echo "Local variable declared inside the function is: ". $num;
   }
   local_var();
?>
```

Output: 45

global variables

The global variables are the variables that are **declared outside the function**.

To access the global variable within a function, use the GLOBAL keyword *before the variable*. However, these variables can be directly accessed or used outside the function without any keyword. Therefore, there is no need to use any keyword to access a global variable outside the function.

```php
<?php
   $name = "Sanaya Sharma";       //Global Variable
   function global_var()
   {
      global $name;
      echo "Variable inside the function: ". $name;
      echo "</br>";
   }
   global_var();
   echo "Variable outside the function: ". $name;
?>
```

Output:

```
Variable inside the function: Sanaya Sharma
Variable outside the function: Sanaya Sharma
```

If two variables, **local** and **global**, have the same name, then the **local variable has higher priority than the global variable inside the function.**

```php
<?php
   $x = 5;
   function mytest()
   {
      $x = 7;
      echo "value of x: " .$x;
   }
   mytest();
?>
```

Output: 7

1. The if statement

syntax for an `if` statement.

```
if(condition){
 //code to execute
}
```

```
<?php
$age = 15;

if ($age < 18) {
 echo "a kid!";
}
?>
```

OUTPUT: `a kid!`

2. if...else statement

PHP if-else statement is executed whether condition is true or false.

```
if(condition){
  //code to execute if condition is true
}
else {
  //code to execute if condition is false
}
```

```
<?php
$age = 18;
if($age < 18){
 echo "a kid!";
}
else {
 echo "so youth!";
}
?>
```

OUTPUT: The output should be: "so youth!". This is because the variable age is not less than 18. If we assign a value like 17 to variable age the output becomes "a kid!"

3. if... else if... else statement

The PHP if-else-if is a **special statement used to combine multiple if?** Else statements. So, we can check multiple conditions using this statement.

```
if (condition) {
  // code to execute if condition is true;
} elseif (condition_2) {
 // code to execute if condition_2 is true
} else {
 // code to execute if none of the above are true
}
```

```
<?php
$age = "18";

if($age < "18"){
 echo "a kid!";
}
elseif($age > "35"){
 echo "a parent?";
}
else{}
 echo"a youth!";
?>
```

OUTPUT

Since the variable `age` is equal to `18` the output of the code above is: `"a youth!"`.

4.The Switch Statement

PHP switch statement is used to execute one statement from multiple conditions. It works like PHP if-else-if statement.

In a switch statement, we provide the deciding factor which can be a variable or an expression to our switch statement, and then we specify the different **cases**, each with a **value**, a piece of code and a **break** statement.

```
switch(expression){
case value1:
 //code to be executed
 break;
case value2:
 //code to be executed
 break;
.....
default:
 code to be executed if all cases are not matched;
}
```

```php
<?php
$car = "Jaguar";

switch($car)
{
    case "Audi":
        echo "Audi is amazing";
        break;
    case "Mercedes":
        echo "Mercedes is mindblowing";
        break;
    case "Jaguar":
        echo "Jaguar is the best";
        break;
    default:
        echo "$car is Ok";
}
?>
```

Output: `Jaguar is the best`

PHP loops

1. PHP For loops

PHP for loop can be used to traverse set of code for the specified number of times.

Syntax

```
For (initialization; condition; increment/decrement) {
//code to be executed
}
```

```php
<?php

for($a = 1; $a <= 10; $a++)
{
    echo "$a <br/>";
}
?>
```

\

Output:

1

2

3

Up to

10

2. PHP While loop

The **while loop executes a block of code repeatedly until the condition is FALSE**. Once the condition gets FALSE, it exits from the body of loop.

It should be used if the number of iterations is not known. The while loop is also called an **Entry control loop** because the condition is checked before entering the loop body.

This means that first the condition is checked. If the condition is true, the block of code will be executed.

Syntax

```
while(condition) {
//code to be executed
}
```

```php
<?php

$a = 1;

while($a <= 10)
{
    echo "$a | ";
    $a++;    // incrementing value of a by 1
}
?>
```

Output: 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

3. PHP do-while loop

The PHP do-while loop is **guaranteed to run at least once.**

If you have to execute the loop at least once and the number of iterations is not even fixed, it is recommended to use the **do-while** loop. It executes the code at **least one time always because the condition is checked after executing the code.**

Syntax

```
Do {
//code to be executed
} while (condition);
```

```php
<?php

$a = 1;

do {
    echo "$a | ";
    $a++;    // incrementing value of a by 1
} while($a <= 10)
?>
```

Output: 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

Let's take another example where even if the condition is false, still the loop will be executed once.

```php
<?php

$a = 11;

do {
    echo $a;
    $a++;    // incrementing value of a by 1
} while($a <= 10)
?>
```

Output: 11

As we can see clearly, that the condition in the above do...while loop will return **false** because value of variable $a is **11** and as per the condition the loop should be executed only if the value of $a is less than or equal to **10**.

7.2 PHP Arrays

PHP Arrays

An **array** is used to store multiple values, generally of same type, in a single variable.

In PHP there are 3 types of array:

1. **Indexed Array**: These are arrays with **numeric index**.
2. **Associative Array**: These are arrays which have a **named key as index**, the key **can be numeric or text.**
3. **Multidimensional Array**: These are **arrays which contain one or more arrays**.

Advantages of Array

Here are a few advantages of using array in our program/script:

1. It's very easy to define simple list of related data, rather than creating multiple variables.
2. It's super easy to use and traverse using the for each loop.
3. PHP provides built-in functions for sorting array, hence it can be used for sorting information as well.

1. PHP Indexed Arrays

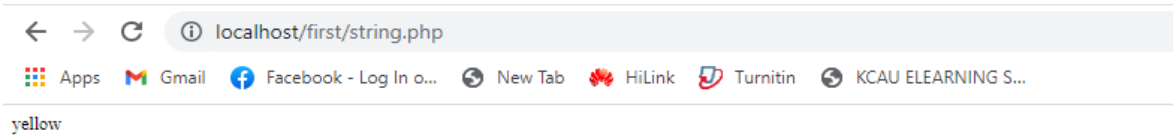
All the array elements are represented by an index which is a numeric value starting from 0 for the first array element.

Example 1

```php
<?php
    $colorArray  = array("Red","yellow","Green " ,"Blue","indigo");
        echo $colorArray [1];

?>
```

Output:



2. Associative Array

These are arrays which have a **named key as index**, the key **can be numeric or text.**

Key-value pair

Associative array **will have their index as string** so that you can establish a strong association with between key and values. Associative elements are passed in the format

" key" =>"value" **=> sign to define key(index) and values**

Example 1 save as associative1.php

```php
<?php
    $marks = array("Maths"=>"85", "Science"=>"89", "English"=>"78");
    echo "Obtained Marks in Maths = ".$marks['Maths'];
?>
```

his is first way to create associative array. We create an array and store in **$marks**.
Here the **key (index)** is user defined, Inside echo statement **$marks['Maths']** is used to fetch the Marks associated with **key** Maths.

In this program "Maths" is act as key (index) of an array, and it also called ***named key***.

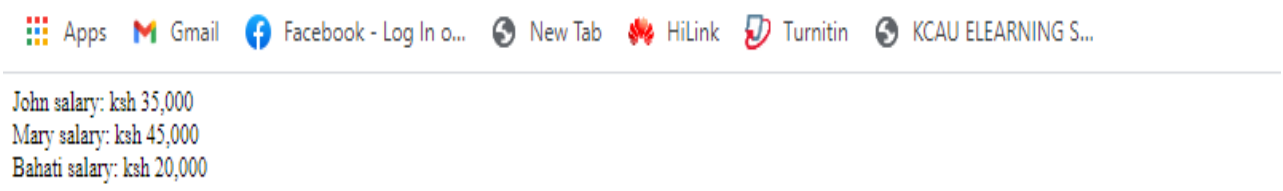
Output:

```
Obtained Marks in Maths = 85
```

Example 2 save as associative2.php

```php
<?php
$salary=array("John"=>"ksh 35,000","Mary"=>"ksh 45,000","Bahati"=>"ksh
    20,000");
echo " John salary: ".$salary["John"]."<br/>";
echo "Mary salary: ".$salary["Mary"]."<br/>";
echo "Bahati salary: ".$salary["Bahati"]."<br/>";
?>
```

Output:



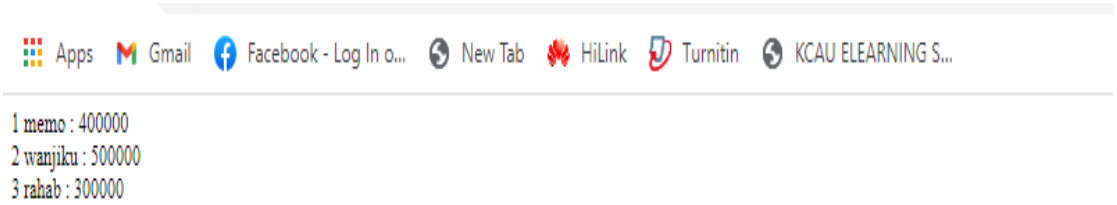
3. Multidimensional Array

PHP multidimensional array is also known as **array of arrays**. It allows you to store tabular data in an array. PHP multidimensional array can be represented in the form of matrix which is represented by row * column.

```php
<?php
$emp = array
  (
  array(1,"memo :",400000),
  array(2,"wanjiku :",500000),
  array(3,"rahab :",  300000),
  );

for ($row = 0; $row < 3; $row++) {
  for ($col = 0; $col < 3; $col++) {
    echo $emp[$row][$col]."  ";
  }
  echo "<br/>";
}
?>
```

Output:



7.3 Working with Forms using HTTP GET & POST methods

HTTP is a protocol that serves as a "**bridge":** it collects a request from the internet browser; sends it to the server; waits for an answer; and, finally, it returns the new information to the browser.

The most used HTTP request types are **GET** and **POST**, but there are other types in their technical specification, such as **PUT, HEAD, DELETE, PATCH** and **OPTIONS**. For the purposes of this class, we will focus only on the two most common.

- **The GET Method**
- **The POST Method**

Before the browser sends the information, it encodes it using a scheme called URL encoding. In this scheme, name/value pairs are joined with equal signs and different pairs are separated by the ampersand.

name1=value1&name2=value2&name3=value3

Spaces are removed and replaced with the + character and any other nonalphanumeric characters are replaced with a hexadecimal value. After the information is encoded it is sent to the server.

The **GET** method sends the **encoded user information appended to the page request**. The page and the encoded information are separated by the**?** character.

http://www.test.com/index.htm?name1=value1&name2=value2

• The GET method produces a long string that appears in your server logs, in the browser's Location: box.

• The **GET** method is **restricted to send up to 1024** (8 KB, or 8192 bytes) characters only. in *URI*

• Never use **GET** method if you have **password** or other **sensitive information** to be sent to the server.

• **GET** can't be used to send **binary data, like images or word documents**, to the server.

• The data sent by **GET** method can be accessed **using QUERY_STRING environment variable.**

• The PHP provides **$_GET** associative array to access all the sent information using GET method.

GET Example Calculator.php

```php
<?php
 if( isset($_GET["name"]) && isset($_GET["age"] ))
 {
 echo "Welcome ". $_GET['name']. "<br />";
 echo "You are ". $_GET['age']. " years old.";
 exit();
 }
?>
<html>
<body>
 <form action="<?php $_PHP_SELF ?>" method="GET">
 Name: <input type="text" name="name" />
 Age: <input type="text" name="age" />
 <input type="submit" />
 </form>
</body>
</html>
```

Output



Code output explanation

The **isset () function** checks whether a variable is set, which means that **it has to be declared and is not NULL.** This functions returns true if the variable exists and is not NULL, otherwise it returns false.

Why use $_Get in PHP is a **super global variable** which is used to collect form data after submitting an HTML form with method**=" get". $ _GET** can also collect data sent in the URL. When a user clicks on the link**" Test $GET,** the parameter "subject**"** and "web" are sent to "test_get.

The POST Method

The **POST** method transfers information via HTTP headers. The information is encoded as described in case of GET method and put into a header called QUERY_STRING.

• The **POST** method **does not** have any **restriction on data size** to be sent.

• The **POST** method can be used **to send ASCII as well as binary data.**

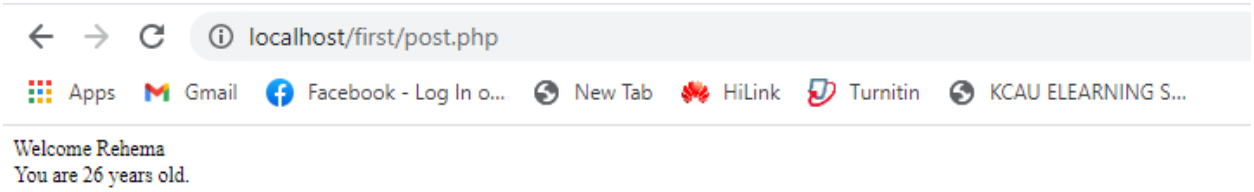
• The data sent by POST method goes through HTTP header so security depends on HTTP protocol. By using Secure HTTP (https) you can make sure that your information is secure.

• The PHP provides $_POST associative array to access all the sent information using GET method.

Example Post.php

```php
<?php
 if( isset($_POST["name"]) && isset($_POST["age"] ))
 {
 echo "Welcome ". $_POST['name']. "<br />";
 echo "You are ". $_POST['age']. " years old.";
 exit();
 }
?>
<html>
<body>
 <form action="<?php $_PHP_SELF ?>" method="POST">
 Name: <input type="text" name="name" />
 Age: <input type="text" name="age" />
 <input type="submit" />
 </form>
</body>
</html>
```

Output:



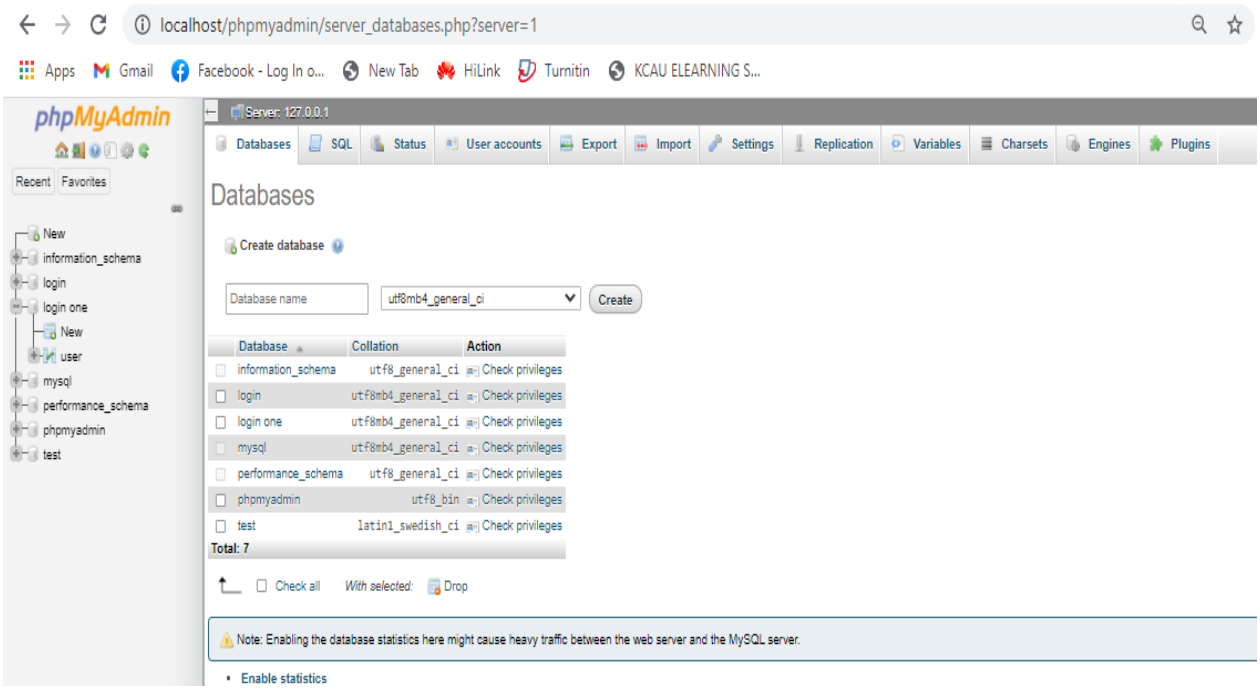
7.4 How to create a login with PHP and MySQL?

How to Create a Login Form with Validation in PHP and MySQL?

How to Create a Login Form with Validation in PHP and MySQL?

Login system using**:**

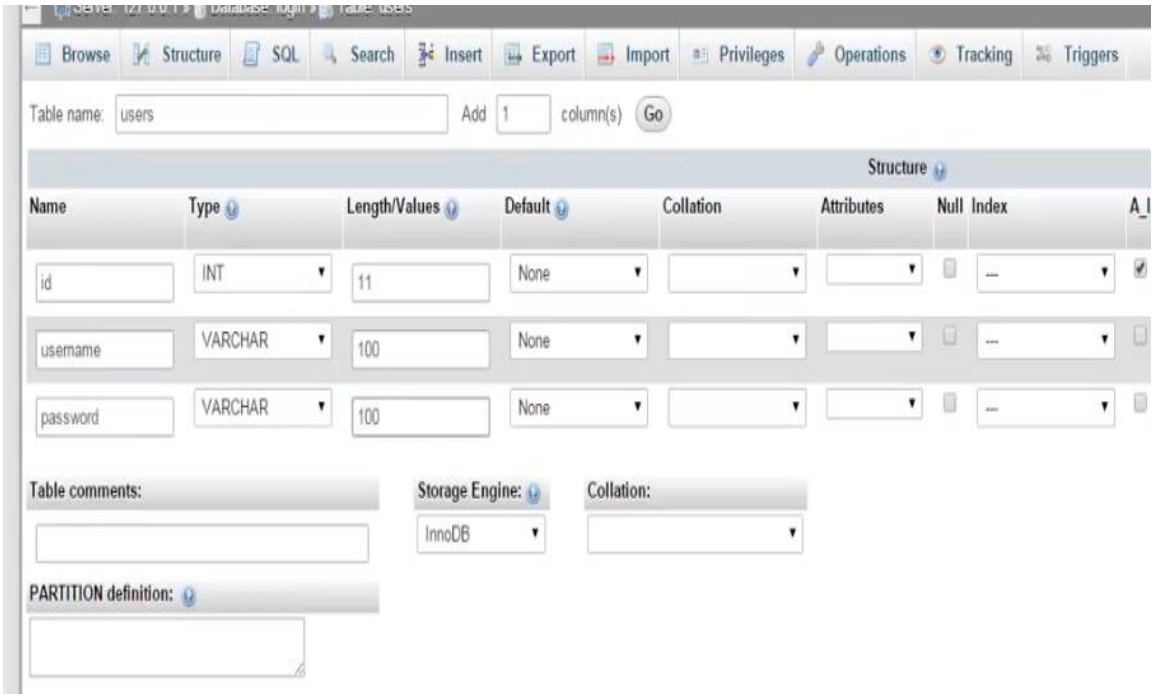
- Html/css
- PHP
- And my sql database
  Step 1: create database using Xampp PHP admin



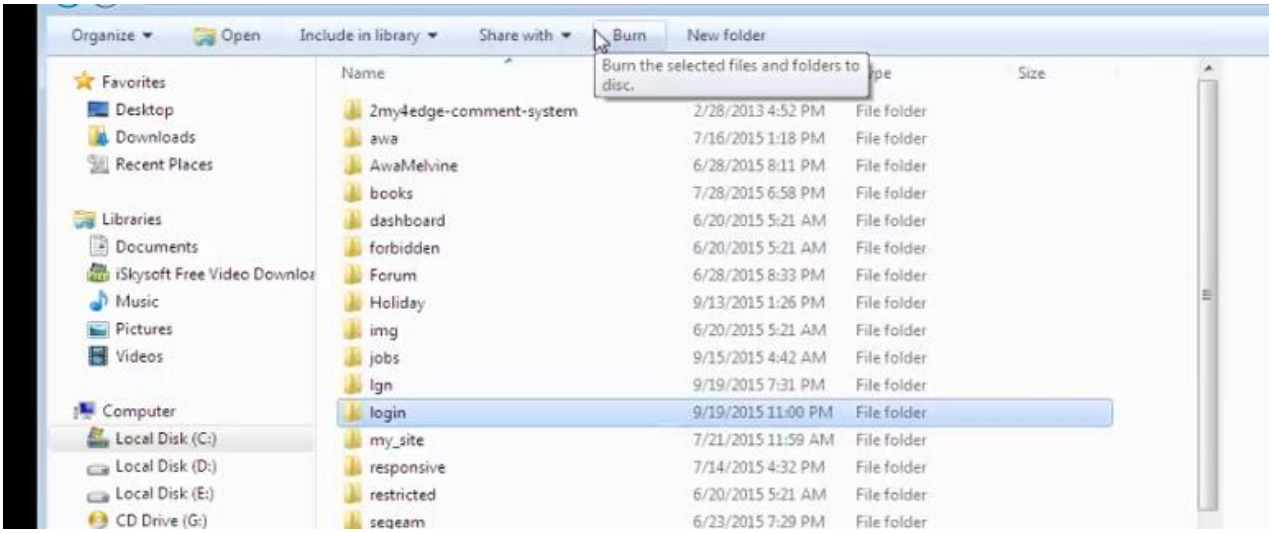
Enter database name as **login** then click **create** button

Step 2. Create table called **user**

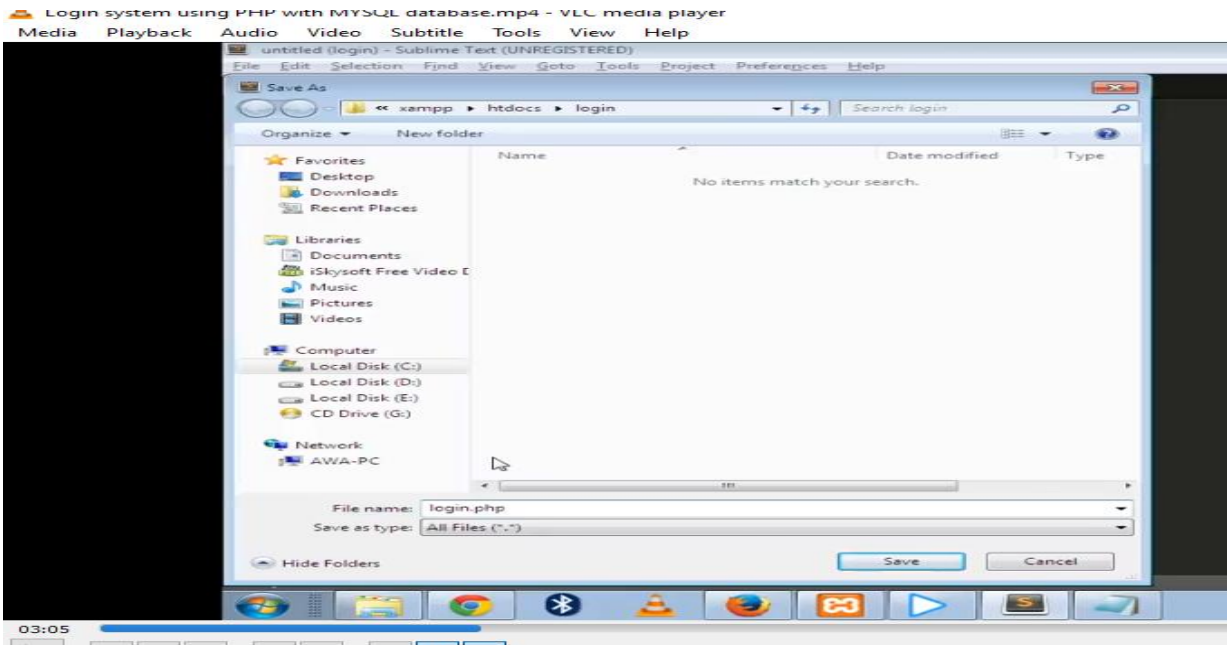
Step 3 create table fields



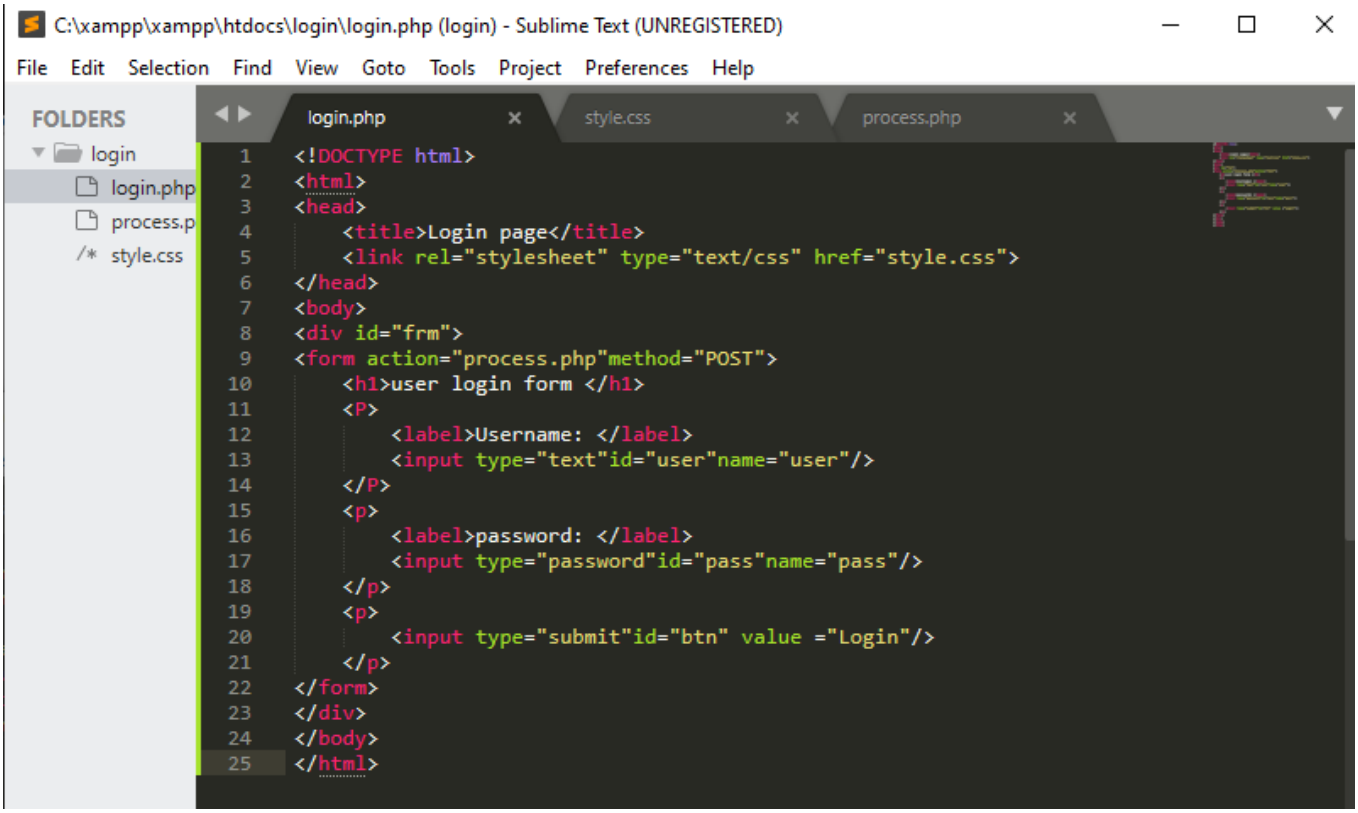
Create path >Xampp>htdocs> login folder



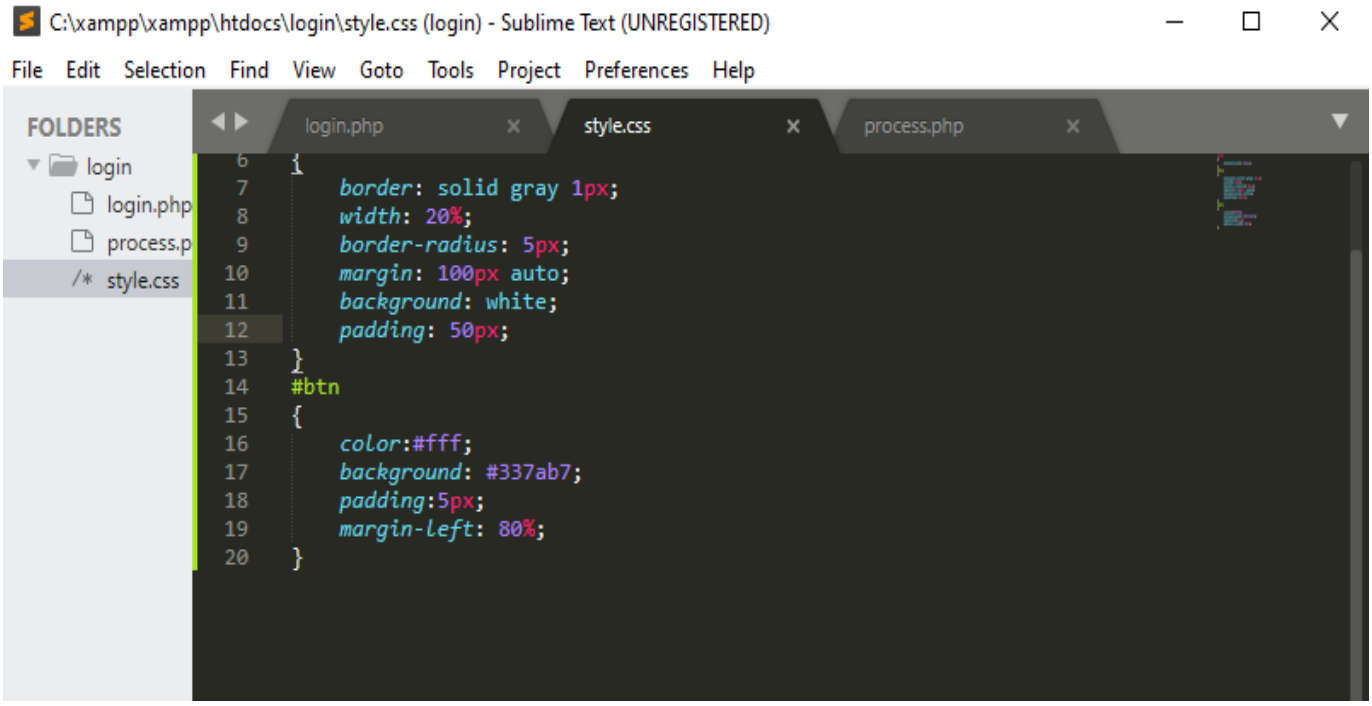
Next step save all table fields inside login folder> open sublime text>save as login.php inside login folder



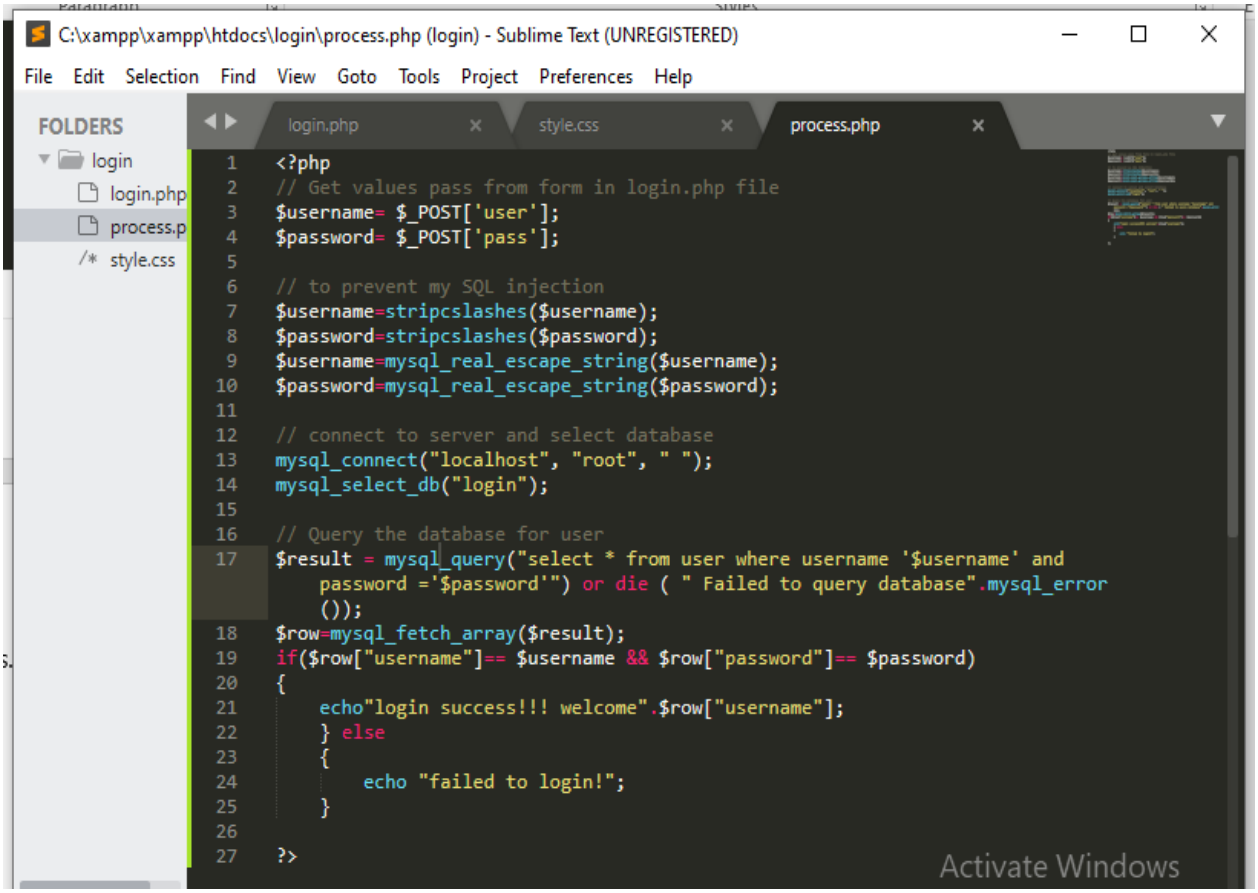
Save as login.php



Next style.css



save as process.php



Output:

Code explanation

Strip slashes () function: > function **removes** backslashes added by the add slashes

This function can be used to clean up a string retrieved from database.

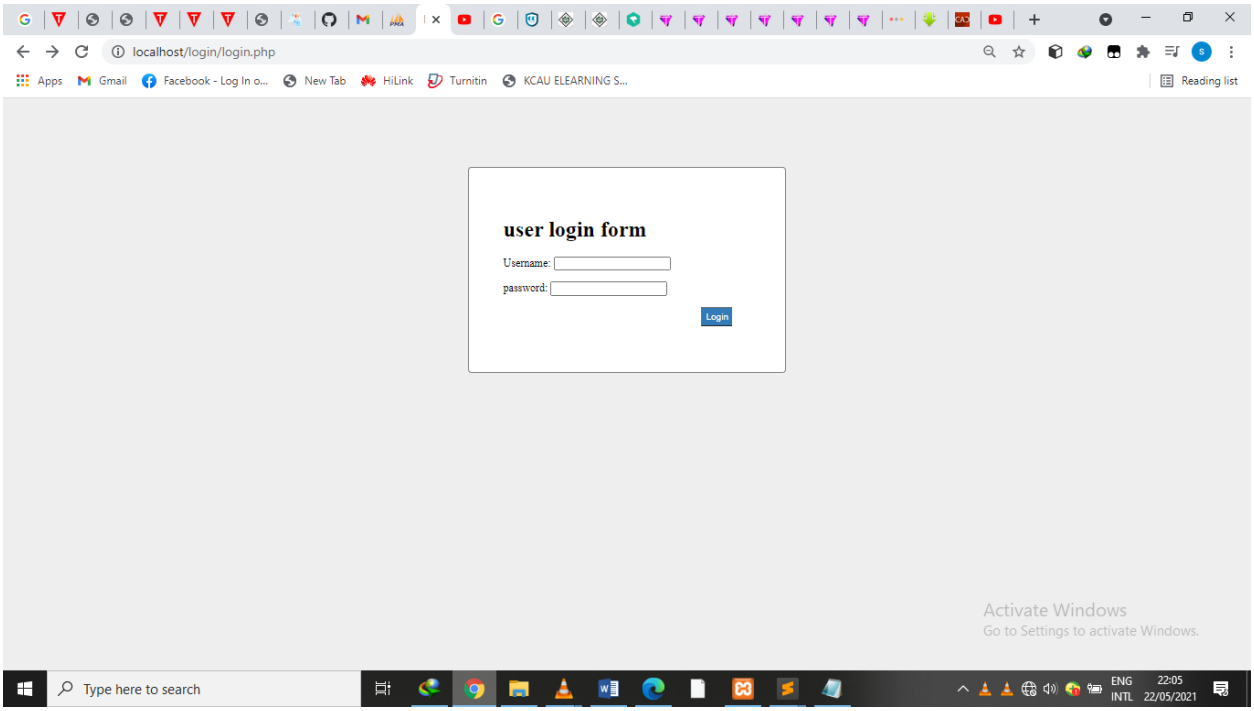
Example

<? Php

Echo string slashes ("Hello\my \ name \is john");

?>

Output: Hello my name is john



Login credentials

Username: usertest

Password: usertestpass

After login

Results successful after data posted from Php form to mysql database.

